iOS SDK Introduction

WeDoBooks SDK Integrator Guide

Setup and Authentication

Goal

Initialize SDK safely, choose backend mode explicitly, and authenticate users before book operations.

Integrator Steps

1. Add SPM dependency from the SDK umbrella repo.
2. Provide a `FirebaseAdapterFactoryType` implementation from FirebaseProvider.
3. Call `WeDoBooksFacade.shared.setup(...)` once per app launch on main actor.
4. Fetch a custom token from your backend and sign in using `userOperations.signIn(with:)`.

Setup Example (Default Firebase App)

```
import WeDoBooksSDK
import FirebaseProvider

@MainActor
func configureSDK() throws {
    try WeDoBooksFacade.shared.setup(
        readerKey: "<reader-key>",
        readerSecret: "<reader-secret>",
        mode: .streaming,
        firebaseAdapterFactory: FirebaseAdapterFactory(),
        firebaseConfigFilePath: nil
    )
}
```

Setup Example (Named Firebase App)

Use a custom plist when the SDK should use a non-default Firebase app.

```
import WeDoBooksSDK
import FirebaseProvider

enum SDKSetupExampleError: Error {
    case missingFirebaseConfigFile(String)
}

@MainActor
func configureSDKWithNamedFirebasePlist() throws {
    let plistName = "GoogleService-Info-WDBSDK"
     guard let configPath = Bundle.main.path(forResource: plistName, ofType: "plist")
else {
        throw SDKSetupExampleError.missingFirebaseConfigFile(plistName)
    }

    try WeDoBooksFacade.shared.setup(
        readerKey: "<reader-key>",
        readerSecret: "<reader-secret>",
```

```
        mode: .streaming,
        firebaseAdapterFactory: FirebaseAdapterFactory(),
        firebaseConfigFilePath: configPath
    )
}
```

Token Acquisition Guidance

- Production: Host app requests token from your backend. Your backend performs backend-to-backend token exchange with the WeDoBooks token endpoint and returns a short-lived custom token to the host app.
- Testing: You can expose a demo token endpoint pattern (for example `https://example-integrator.test/wedobooks/demo-token`) that returns a token for sandbox testing flows.

Security note: Demo token endpoints are for testing and non-production flows only.

Auth Example (Sign In With Custom Token)

```
func signIn(token: String) async {
    let result = await WeDoBooksFacade.shared.userOperations.signIn(with: token)
    if case .failure(let error) = result {
        print("Sign in failed: \(error)")
    }
}
```

Flow Diagram

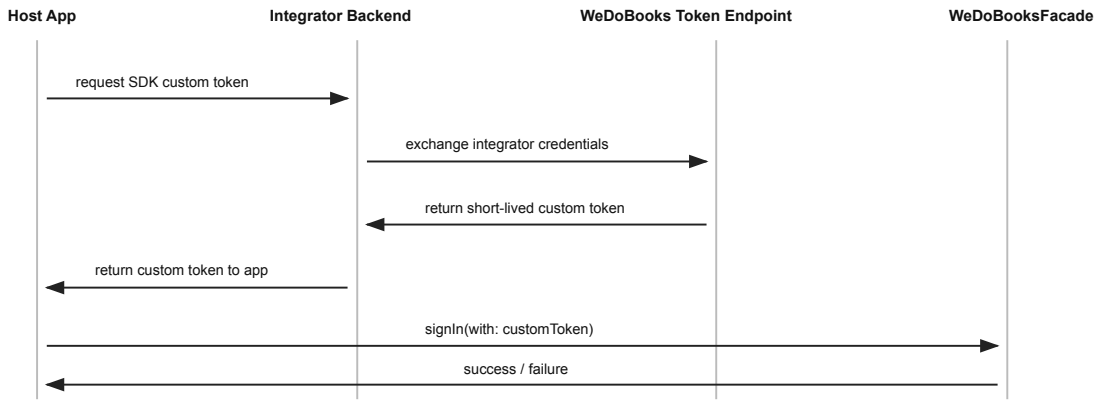

Figure 1: Setup and auth sequence

Optional Demo Token Fetch Example (Testing)

```
import Foundation

struct DemoTokenResponse: Decodable {
    let token: String
}
```

```
func fetchDemoTokenForTesting(userId: String) async throws -> String {
    let encodedUser = userId.addingPercentEncoding(
        withAllowedCharacters: .urlQueryAllowed
    ) ?? userId

    let baseURL = "https://example-integrator.test/wedobooks/demo-token"
    let url = URL(string: "\(baseURL)?user_id=\(encodedUser)")!
    let (data, _) = try await URLSession.shared.data(from: url)

    return try JSONDecoder()
        .decode(DemoTokenResponse.self, from: data)
        .token
}
```

Notes

- `setup(...)` is intentionally explicit about backend mode.
- Repeated setup attempts throw `SetupError.setupAlreadyCalled`.
- Most namespaces that interact with user data require a signed-in user.

Device Management Example

Use `devicesManager.observeDevices` to monitor devices and `devicesManager.removeDevices(with:)` to remove one or more device ids from the current user's device list.

```
func removeExistingDevice(deviceId: String) async {
        let result = await WeDoBooksFacade.shared.devicesManager.removeDevices(with:
[deviceId])
    if case .failure(let error) = result {
        print("Could not remove device: \(error)")
    }
}
```

Limits and Active Device

The backend currently enforces a per-user device limit of **6 registered devices**. When the limit is reached, the next call to `bookOperations.openCheckout(...)` or `headlessAudioPlayer.loadBook(...)` throws `DeviceManagementError.deviceLimitReached`; call `devicesManager.removeDevices(with:)` to free a slot.

Only one device can be active per user at any time. When another device becomes active, the SDK terminates the local reader/player/headless session and emits `.sessionTakenOver` on `events.sessionInterrupted`.

Cross References

- DocC: GettingStarted.md
- DocC symbol root: `WeDoBooksFacade`

Checkout and Open

Goal

Request/observe user checkouts and present reader/player UI safely.

Recommended Flow

1. Ensure user is signed in.
2. Observe checkouts (`observeCheckouts`) for UI list updates.
3. Request missing checkout (`checkoutBook(with:)`) when needed.
4. Present using `openCheckout(_:presentedBy:customCover:)` on main actor.

Checkout + Open Example

```
func open(isbn: String, presenter: UIViewController) async {
    let result = await WeDoBooksFacade.shared.bookOperations.checkoutBook(with: isbn)
    switch result {
    case .success(let checkout):
        do {
                try await WeDoBooksFacade.shared.bookOperations.openCheckout(checkout,
presentedBy: presenter)
        } catch {
            print("Open failed: \(error)")
        }
    case .failure(let error):
        print("Checkout failed: \(error)")
    }
}
```

Notes

- Only one SDK book UI can be open at a time.
- If headless audiobook playback is active, `openCheckout` throws `OpenCheckoutError.aBookIsAlreadyOpen`.
- `openCheckout` can throw `DeviceManagementError.deviceLimitReached`; free a slot via `devicesManager.removeDevices(with:)` and retry.
- Ebook open path may show download/progress UI before reader when local file is missing.

Samples

`openSample(for:format:presentedBy:)` is the single entry point for presenting either the SDK reader or audio player on a sample without an active checkout. Pass `MaterialType.ebook` or `MaterialType.audiobook` as the format. The call is `async throws`: synchronous pre-flight guards (signed-in user, no other SDK book UI open, valid `materialId`) throw before the first `await`; the URL fetch (and EPUB download for `.ebook`) is awaited inline and any failure is re-thrown to the caller.

```
Task { @MainActor in
    do {
        try await WeDoBooksFacade.shared.bookOperations.openSample(
            for: materialId,
            format: .audiobook,
```

```
            presentedBy: presenter
        )
    } catch let error as SampleError {
        // .invalidMaterialId, .unauthenticated, .aBookIsAlreadyOpen,
        // .notFound, or .underlyingError(...)
        print("Sample open failed: \(error)")
    }
}
```

`format: .ebook` skips progress persistence, `lastOpened` updates, reading-time statistics, the finish-book button, and bookmarks in full (text-selection menu item, existing bookmark annotations, chapter-overview Bookmarks segment, and bookmark writes are all suppressed). Samples always open at the beginning even when the user has saved progress on the same material's full book.

`format: .audiobook` skips progress persistence, `lastOpened` updates, audible-time statistics, Now Playing info, and remote-control event reception. In the player UI the sleep-timer button, the finish-book button, and the more menu are hidden. The audio player UI has no bookmark or table-of-contents surfaces, so no additional bookmark suppression is needed at the UI layer. Samples always start at position 0.

`openSample` performs the same backend device activation as `openCheckout` before presenting any sample UI: this device becomes the user's active device and any other device with a live SDK session is taken over (terminated and surfaced through `events.sessionInterrupted` as `.sessionTakenOver`). `openSample` can therefore also throw `DeviceManagementError.deviceLimitReached` (or `DeviceManagementError.underlyingError(_)`) — free a slot via `devicesManager.removeDevices(with:)` and retry.

The `materialId` is rejected if it is empty, not exactly 13 characters, or contains `/`, `\`, or `.` — this guards the on-disk temporary filename used for ebook samples (which embeds the `materialId` directly). Standard ISBN-13 identifiers satisfy this constraint.

Any `FirebaseError` (or empty/invalid URL response) is classified as the retryable `SampleError.notFound` via `SampleError.from(_:)`; other errors become `SampleError.underlyingError(_:)`. The arbitration check is repeated on the main actor after the URL fetch resolves, so a concurrent SDK open can still throw `SampleError.aBookIsAlreadyOpen` after the await.

The headless `headlessAudioPlayer.loadSample(for:)` API remains available for integrators that want to drive audiobook sample playback without presenting SDK UI.

Cross References

- DocC: BookOperations.md

Offline and Storage

Goal

Help host apps let users download ebooks and audiobooks, show useful offline state, recover from storage failures, and understand what happens when downloaded material is removed or a user signs out.

Offline support is centered around a `Checkout`. A checkout tells the SDK which material belongs to the signed-in user, and the storage APIs use that checkout to download, inspect, and remove local material.

Offline Model

Downloaded material is scoped to the current signed-in user. The SDK stores downloaded ebook/audio artifacts in app storage, keeps offline bookmarks and related metadata in local preferences, and stores ebook offline keys in the keychain.

Ebooks are always downloaded before they are opened so they are available offline immediately after that and until the material is removed explicitly through the API or the user logs out. Audiobooks can be downloaded through a call to the API. After download has completed successfully then the books can also be opened when offline.

API Quick Reference

Use these APIs to build host UI around the offline model:

- `storageOperations.download(book:)`: Start downloading a checkout for offline use.
- `storageOperations.downloadStatus(book:)`: Render download state and progress in host UI.
- `storageOperations.isBookDownloaded(isbn:)`: Check whether the current user has local material for an ISBN.
- `storageOperations.getAllDownloadedBooks()`: List ISBNs with local offline records for the current user.
- `storageOperations.removeDownload(isbn:)`: Remove one downloaded material item for the current user.
- `storageOperations.removeAll()`: Clear SDK local offline/download storage.
- `storageOperations.onError`: Observe asynchronous download/storage failures.
- `storageOperations.downloadStatuses`: Observe asynchronous download status updates.
- `userOperations.signOut()`: Sign out and clear SDK local offline/download state.

`downloadStatus(book:)` is usually the best API for UI because it can distinguish between in-progress downloads, completed downloads, and material that is not downloaded. `downloadStatuses` publishes snapshots keyed by material identifier (represented as ISBN in current integrator flows).

Most storage APIs require setup to have completed and a user to be signed in. Calls that cannot even start because setup or sign-in is missing throw synchronously. Download failures that happen after a request has started are reported asynchronously through `storageOperations.onError`.

Download Lifecycle

Start by acquiring a `Checkout`, for example through `bookOperations.checkoutBook(with:)` or `bookOperations.observeCheckouts()`. Pass that checkout to `download(book:)`.

```
import Combine
import WeDoBooksSDK

private var cancellables = Set<AnyCancellable>()
```

```
func prepareOfflineAccess(for checkout: Checkout) {
    WeDoBooksFacade.shared
        .storageOperations
        .onError
        .sink { error in
            print("Offline download failed: \(error)")
        }
        .store(in: &cancellables)

    do {
        try WeDoBooksFacade.shared
            .storageOperations
            .download(book: checkout)
    } catch {
        print("Could not start download: \(error)")
    }
}
```

If the material already exists on device for the current user, `download(book:)` throws `StorageOperationError.alreadyDownloaded` instead of starting a new download. Treat this as a benign, already-satisfied state rather than a failure (for example, surface “already available offline” rather than an error).

Use `downloadStatus(book:)` to keep host UI in sync:

```
func refreshOfflineState(for checkout: Checkout) {
    do {
        let status = try WeDoBooksFacade.shared
            .storageOperations
            .downloadStatus(book: checkout)

        switch status {
        case .initializing:
            print("Preparing download")
        case .downloading(let progress):
            print("Downloading \(Int(progress * 100))%")
        case .downloaded:
            print("Available offline")
        case .notDownloaded:
            print("Not downloaded")
        case .failure:
            print("Download failed")
        case .cancel:
            print("Download canceled")
        }
    } catch {
        print("Could not read download status: \(error)")
```

```
    }
}
```

Or observe download statuses:

```
func observeDownloadStatuses() {
    WeDoBooksFacade.shared.storageOperations.downloadStatuses
        .sink { statusesByISBN in
            print("Known download statuses: \(statusesByISBN)")
        }
        .store(in: &cancellables)
}
```

Status values mean:

- `.initializing`: a download has been requested and is being prepared.
- `.downloading(progress:)`: a download is active. `progress` is a `0.0...1.0` fraction.
- `.downloaded`: local material exists for the current user and checkout.
- `.notDownloaded`: local material is absent, not started, removed, or the latest start attempt failed before a usable download state was established.
- `.failure`: the latest download attempt failed.
- `.cancel`: the latest download attempt was canceled.

To check availability without rendering progress, use:

```
func printDownloadedISBNs() {
    do {
        let isbns = try WeDoBooksFacade.shared
            .storageOperations
            .getAllDownloadedBooks()
        print("Downloaded ISBNs: \(isbns)")
    } catch {
        print("Could not list downloads: \(error)")
    }
}
```

Opening Downloaded Material

Open a checkout the same way whether it has been downloaded or not:

```
@MainActor
func open(_ checkout: Checkout, from presenter: UIViewController) async {
    do {
        try await WeDoBooksFacade.shared
            .bookOperations
            .openCheckout(checkout, presentedBy: presenter)
    } catch {
```

```
        print("Could not open checkout: \(error)")
    }
}
```

For ebooks, offline opening requires both the downloaded file and the SDK's stored offline decryption metadata. If that local information has been removed or is invalid, opening can fail with `OpenCheckoutError.decryptionInformationInvalidOrMissing`. In that case, guide the user to download or open the material again while online.

Disk Space and Download Failures

The public storage API does not expose a dedicated "disk full" error case. If the device runs out of space or the filesystem cannot persist downloaded material, the host app should expect a `StorageOperationError.underlyingError` through `storageOperations.onError` or as a thrown error from a storage operation, depending on where the failure occurs.

A practical recovery path is:

1. Tell the user the download could not be completed.
2. Suggest freeing device storage.
3. Let the user retry the download.
4. Refresh `downloadStatus(book:)` after the failure or retry.

```
func handleStorageError(_ error: StorageOperationError) {
    switch error {
    case .setupNotCompleted:
        print("SDK setup has not completed.")
    case .noUserSignedIn:
        print("Sign in before using offline storage.")
    case .downloadNotAllowedOnCellular:
        print("Connect to Wi-Fi or allow cellular downloads.")
    case .alreadyDownloaded:
        print("This book is already available offline.")
    case .underlyingError:
        print("The download failed. Free up space and try again.")
    }
}
```

Removing Downloaded Material

Use `removeDownload(isbn:)` to remove a single downloaded item for the current user:

```
func removeOfflineMaterial(isbn: String) {
    do {
        try WeDoBooksFacade.shared
            .storageOperations
            .removeDownload(isbn: isbn)
    } catch {
        print("Could not remove downloaded material: \(error)")
```

```
    }
}
```

After a successful removal, the SDK clears the cached download state for that ISBN. A later `downloadStatus(book:)` call should resolve to `.notDownloaded`.

Opening the same material again after removal may require network access so the SDK can stream or download the material again. For ebooks, the removed offline file and decryption metadata cannot be used after removal.

Use `removeAll()` when the host app needs to clear SDK offline storage explicitly:

```
func clearAllSDKOfflineStorage() {
    WeDoBooksFacade.shared
        .storageOperations
        .removeAll()
}
```

`removeAll()` clears downloaded artifacts, bookmarks/offline metadata, ebook offline data, easy-access entries, and cached download-status state.

Logout Behavior

`userOperations.signOut()` signs out the current user and triggers SDK cleanup of local offline storage.

```
func signOut() {
    WeDoBooksFacade.shared
        .userOperations
        .signOut()
    // SDK local downloaded material and offline metadata are cleared.
}
```

After logout, integrators should expect downloaded material, offline metadata, easy-access data, and cached download-status state to be cleared. A newly signed-in user must establish their own checkout and download state.

Implementation Notes

- Download failures can be synchronous (`throw`) or asynchronous (`onError`).
- Status is exposed as `StorageDownloadStatus`: `initializing`, `downloading(progress:)`, `downloaded`, `notDownloaded`, `failure`, or `cancel`.
- `initializing`, `failure`, and `cancel` are public enum cases. Integrations with exhaustive `switch` statements over `StorageDownloadStatus` must handle them.
- `downloadStatuses` emits the current snapshot immediately for the current user's persisted downloads plus active/known download states, then emits whenever a known book's state or progress changes.
- `downloadStatuses` is scoped to the current signed-in user. Switching users resets in-session download tracker state before the new user's snapshot is emitted.
- User sign-out clears relevant local storage state via facade delegate flow.

Cross References

- DocC: StorageOperations.md
- DocC: BookOperations.md
- DocC: UserOperations.md

Customization

Goal

Adjust SDK behavior and visual integration without forking SDK internals.

Available Areas

- `configuration`: feature toggles and persisted behavior flags
- `styling`: light/dark theme override
- `localization`: language selection and string overrides
- `images`: icon and reader image overrides
- `events`: host-side reactions to SDK UI actions

Configuration Example

```
let facade = WeDoBooksFacade.shared
facade.configuration.showFinishEbookButton = true
facade.configuration.showFinishAudiobookButton = true
facade.configuration.showAboutAudioBookButton = true
facade.configuration.showAudiobookMinimizeButton = true
```

Localization Override Example

```
typealias LocalizationOverrides = [
    WeDoBooksFacade.Localization.LocalizationKeys: [
        WeDoBooksFacade.Localization.Language: String
    ]
]

let overrides: LocalizationOverrides = [
    .buttonSave: [.english: "Save"],
    .playerPlaybackRateSpeed: [.english: "Speed"]
]

WeDoBooksFacade.shared.localization.setCustomLocalizations(overrides)
```

Events Example

```
private var cancellables: Set<AnyCancellable> = []

func observeEvents() {
    WeDoBooksFacade.shared.events.finishBookTapped
        .sink { isbn in
            print("Finish tapped for \(isbn)")
        }
        .store(in: &cancellables)
}
```

Cross References

- DocC: Configuration.md
- DocC: Styling.md
- DocC: Localization.md
- DocC: Images.md
- DocC: Events.md

Headless Audio and CarPlay

Goal

Use SDK audiobook runtime without presenting SDK player UI, and optionally integrate transport controls in host app surfaces.

Headless Flow

1. Ensure setup + sign-in is completed.
2. Load audiobook checkout with `headlessAudioPlayer.loadBook(...)`.
3. Use `play`, `pause`, `seek`, `setRate` controls.
4. Observe `onStateChange` and `onError` on main queue.
5. Observe `events.sessionInterrupted` to react when another device takes over the session.

`loadBook(...)` also updates the device-local easy-access pointer for the loaded checkout (the same pointer SDK player UI and the reader set), so the host's easy-access widget (`easyAccess.lastOpenedBook()`) reflects the headless book. `loadSample(...)` does not.

Headless Example

```
func startHeadless(_ checkout: Checkout) async {
    do {
        try await WeDoBooksFacade.shared
            .headlessAudioPlayer
            .loadBook(
                book: checkout,
                startPosition: 0
            )

        try WeDoBooksFacade.shared
            .headlessAudioPlayer
            .play()
    } catch let error as DeviceManagementError {
        // device activation failure, e.g. .deviceLimitReached
        print("Headless activation failed: \(error)")
    } catch {
        print("Headless failed: \(error)")
    }
}
```

CarPlay Example

```
func wireNowPlaying() {
    _ = WeDoBooksFacade.shared.carPlay.nowPlayingInfo
        .sink { info in
            print("Now playing info changed: \(String(describing: info))")
        }
}
```

```
func observeSessionInterruption() {
    _ = WeDoBooksFacade.shared.events.sessionInterrupted
        .sink { event in
            if event.reason == .sessionTakenOver {
                print("Session moved to device: \(event.activeDevice)")
            }
        }
}
```

Sample Playback

`loadSample(for:)` loads an audiobook sample into the headless runtime. It takes a raw `materialId` (not a `Checkout`) and never accepts a caller-supplied start position; samples always begin at 0. The call is `async throws` so backend device activation can be awaited in line with `loadBook`.

```
func startSample(materialId: String) {
    Task { @MainActor in
        do {
         try await WeDoBooksFacade.shared.headlessAudioPlayer.loadSample(for: materialId)
            try WeDoBooksFacade.shared.headlessAudioPlayer.play()
        } catch let error as HeadlessAudiobookError {
            // .invalidMaterialId, .setupNotCompleted, .noUserSignedIn,
            // .playerControlledByPresentedSDKUI, .audiobookAlreadyPlaying,
            // or .sameBookAlreadyLoaded
            print("Sample load rejected: \(error)")
        } catch let error as DeviceManagementError {
            // .deviceLimitReached or .underlyingError(_)
            print("Sample device activation failed: \(error)")
        } catch {
            print("Sample load failed: \(error)")
        }
    }
}
```

Sample sessions suppress Now Playing info, remote control event registration, bookmark writes, `loanService.updateLastOpened` updates, the easy-access pointer write (only `loadBook` sets it), and reading-time stats.

`loadSample` performs the same backend device activation as `loadBook` before reserving the loaded slot: this device becomes the user's active device and any other device with a live SDK session is taken over (terminated and surfaced through `events.sessionInterrupted` as `.sessionTakenOver`). `loadSample` can therefore also throw `DeviceManagementError.deviceLimitReached` (or `DeviceManagementError.underlyingError(_)`) — free a slot via `devicesManager.removeDevices(with:)` and retry.

The URL fetch is asynchronous and runs after activation succeeds; fetch failures surface on `onError` as `HeadlessAudiobookError.underlyingError(SampleError)`. Any `FirebaseError` is classified as the retryable `SampleError.notFound`; the loaded slot is reset on failure so subsequent `loadSample` / `loadBook` calls aren't blocked.

The `materialId` is rejected if empty or containing `/`, `\`, or `..`.

Notes

- Headless and SDK Player UI share the same underlying runtime.
- Runtime ownership collisions are surfaced as deterministic errors.
- Session takeover on another device terminates the active SDK reading/playback session (reader UI, player UI, or headless) and emits `events.sessionInterrupted`.
- On session activation, SDK registers this device before marking it active; if the device already exists in backend registry, SDK still marks it active.
- CarPlay controls are non-throwing helpers over the shared runtime.

Cross References

- DocC: HeadlessAudioPlayer.md
- DocC: CarPlay.md

Error Handling and Troubleshooting

Goal

Provide quick diagnosis paths for common setup/auth/open/storage/headless failures.

Error Families

- `SetupError`: setup lifecycle and reader initialization.
- `SignInError`: auth and token/factory integration issues.
- `CheckoutError` / `OpenCheckoutError` / `DeviceManagementError`: checkout/open/presentation and activation failures.
- `StorageOperationError`: offline and download issues.
- `HeadlessAudiobookError`: headless runtime state and arbitration errors.
- `SampleError`: sample reader/playback failures. `.invalidMaterialId` (empty or contains `/`, `\`, `.`), `.unauthenticated` (no signed-in user), `.aBookIsAlreadyOpen` (collision with active SDK UI / headless playback), `.notFound` (backend missing or transient internal failure — retryable), `.underlyingError(_:)` (non-Firebase failure path). Additive — host apps with exhaustive `switch` handling without `@unknown default` must include this case.

Troubleshooting Matrix

Use this as a quick lookup when mapping SDK errors to host-app diagnostics:

- `setupAlreadyCalled`: setup executed more than once. First check app lifecycle and singleton setup location.
- `firebaseNotConfigured`: Firebase app config is missing or invalid. First check adapter factory and config plist path.
- `noUserSignedIn`: user-dependent API was called before sign-in. First check the auth token flow and sign-in result handling.
- `aBookIsAlreadyOpen`: runtime ownership collision. First check for existing SDK player/reader UI or headless playback.
- `deviceLimitReached`: device activation failed during open. The backend currently enforces a limit of 6 registered devices per user, and only one device can be active at a time. First inspect the user's device list via `devicesManager.observeDevices` and call `removeDevices(with:)` to free a slot.
- `downloadNotAllowedOnCellular`: cellular download policy blocked the request. First check `configuration.allowEbookDownloadUsingMobileData`.
- `alreadyDownloaded`: `storageOperations.download(book:)` was called for material already present on device for the current user. This is benign; check `isBookDownloaded(isbn:)` / `downloadStatus(book:)` before offering a download action.

Diagnostics Checklist

1. Confirm `setup(...)` succeeded this launch.
2. Confirm expected backend mode (`streaming` or `library`).
3. Confirm current user id is non-nil before user-scoped APIs.
4. Confirm runtime ownership before open/headless transitions.
5. Confirm network/download policy assumptions.

Observing Background Errors (support and diagnostics only)

`WeDoBooksFacade.shared.errors` is a firehose that mirrors Firebase-layer failures (every cloud-function callable failure plus Firestore background-listener failures). It exists **only for support and diagnostics** — subscribe to forward what it emits to your logging, crash-reporting, or support tooling:

```
WeDoBooksFacade.shared.errors
    .receive(on: DispatchQueue.main)
    .sink { error in /* forward to logging / crash reporting */ }
    .store(in: &cancellables)
```

It is **not** a programmatic error channel: do not drive error recovery, retries, control flow, or user-facing UI from it — the typed errors returned by the individual APIs above are the only caller-actionable channel.

The stream is replay-less (subscribe early so you don't miss any), is delivered on the thread the error was reported on (typically a background queue — apply `.receive(on:)` yourself if your logging requires the main thread), and emits `any Error` — treat each error as opaque (log it) rather than switching on its concrete type. It is a superset of, not a replacement for, the typed errors returned by the individual APIs above — a callable failure is mirrored here *and* returned by its typed API, so dedupe if you log from both.

Cross References

- DocC: ErrorHandling.md

History

Goal

Display a user's reading/listening history, check per-material status, and add/complete/remove entries programmatically.

Requirements

- SDK configured with `.streaming` mode. All `historyOperations` methods return `.failure(.unsupportedInLibraryMode)` in `.library` mode.
- A signed-in user. Without one all methods return `.failure(.noUserSignedIn)`.

Listing History

```
let result = await WeDoBooksFacade.shared.historyOperations.list(
    filter: .all,         // .all | .inProgress | .completed
    sort: .newestActivity, // .newestActivity | .oldestActivity
    after: nil,           // pass HistoryCursor from previous page for pagination
    limit: 25
)
switch result {
case .success(let page):
    for item in page.items {
        print("\(item.title) — \(item.status)")
    }
case .failure(let error):
    print("History list failed: \(error)")
}
```

Pagination

When `page.nextCursor` is non-nil, more entries exist. Pass the cursor back as `after:` to load the next page.

```
var cursor: HistoryCursor? = nil
var allItems: [HistoryItem] = []

repeat {
    let result = await WeDoBooksFacade.shared.historyOperations.list(
        after: cursor,
        limit: 25
    )
    guard case .success(let page) = result else { break }
    allItems.append(contentsOf: page.items)
    cursor = page.nextCursor
} while cursor != nil
```

> **Note:** `HistoryCursor` is session-scoped (snapshot-based). Discard it on app restart; it is not serializable across launches.

Checking a Single Material's Status

Use `entry(for:)` to show an "In Progress" or "Completed" badge on a product page. Returns `nil` when the material is not in history.

```
let result = await WeDoBooksFacade.shared.historyOperations.entry(for: materialId)
if case .success(let item) = result, let item {
    print("Status: \(item.status), progress: \(item.progress)")
}
```

Adding an Entry Manually

`add(materialId:)` creates a Completed entry. Fails with `.alreadyInHistory` when the material is already present, or `.materialNotFound` when the material does not exist in the catalogue.

```
let result = await WeDoBooksFacade.shared.historyOperations.add(materialId: isbn)
switch result {
case .success:
    print("Added to history")
case .failure(.alreadyInHistory):
    print("Already in history — no action needed")
case .failure(.materialNotFound):
    print("Material not found")
case .failure(let error):
    print("Add failed: \(error)")
}
```

Marking an Entry Completed

Idempotent. Returns `.notInHistory` if the material has no history entry.

```
let result = await WeDoBooksFacade.shared.historyOperations.markCompleted(materialId:
isbn)
if case .failure(let error) = result {
    print("Mark completed failed: \(error)")
}
```

Removing an Entry

Hard-deletes the history entry. Reading progress (`book_progress`) is preserved — if the user re-opens the book, the backend creates a fresh history entry. Returns `.notInHistory` if no entry exists.

```
let result = await WeDoBooksFacade.shared.historyOperations.remove(materialId: isbn)
if case .failure(let error) = result {
    print("Remove failed: \(error)")
}
```

Notes

- `progress` is a `Double` in the range 0–1. Compute percentage, time remaining, or pages remaining in the host app from `progress` combined with `durationSeconds` (audiobook) or `wordCount` (ebook).

Error Reference

Error	Trigger
`.noUserSignedIn`	No authenticated user
`.unsupportedInLibraryMode`	SDK in `.library` mode
`.alreadyInHistory`	`add` — material already in history
`.materialNotFound`	`add` — material not in catalogue
`.notInHistory`	`markCompleted` / `remove` — no history entry
`.underlyingError(Error)`	Network or Firestore failure

Cross References

- DocC: History.md